

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0518

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Face Detection and Recognition Pipeline

A system must process a live video stream to:

- A. Capture frames from a camera
- B. Detect faces in each frame
- C. Recognize faces using a pre-trained dataset
- D. Display bounding boxes and labels in real time

Constraints:

- Must operate continuously
- Should minimize processing delay
- Should handle multiple faces

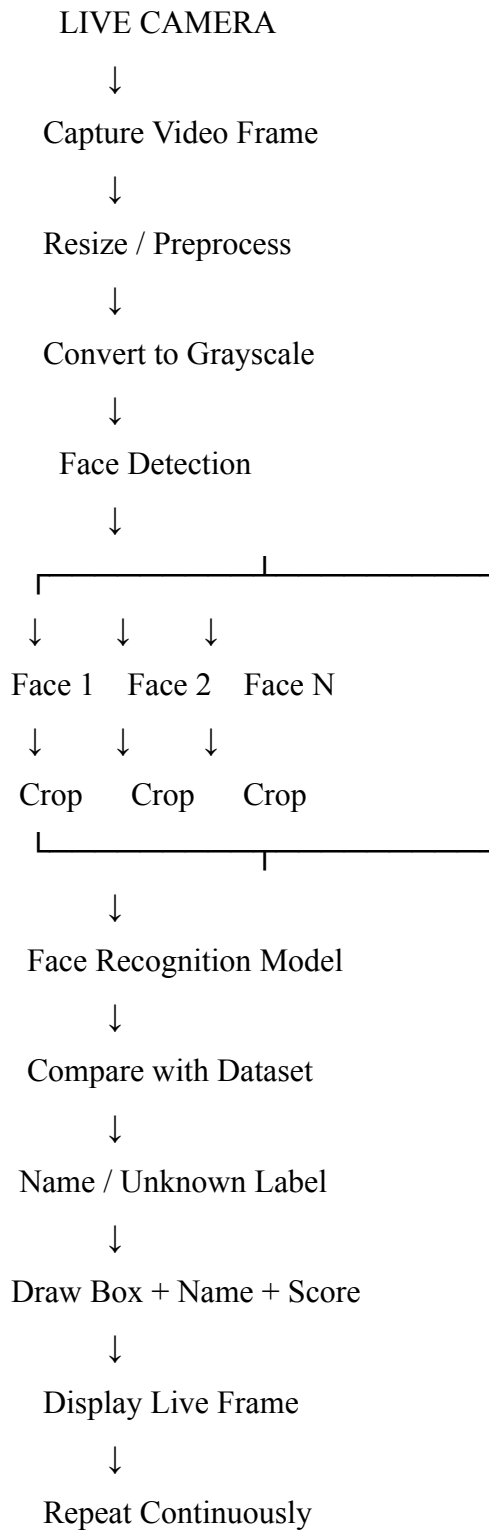
Write detailed pseudocode for the complete pipeline using OpenCV concepts, including image acquisition, preprocessing, detection, recognition, and display. Ensure modular design and efficient looping structure.

ANSWER:

1. Aim

To design a real-time OpenCV-based face detection and recognition system that captures video from a camera, detects multiple faces, recognizes them using a pre-trained dataset, and displays bounding boxes with names continuously.

2. Block Diagram



3. Detailed Pseudocode

BEGIN

1. INITIALIZE SYSTEM

Import OpenCV

Load pre-trained face detection model

Load trained face recognition model

Load trained face dataset / face embeddings

Load corresponding person names

Initialize camera:

cap = cv2.VideoCapture(0)

Set camera resolution

Set recognition confidence threshold

2. DEFINE capture_frame()

Read frame from camera

IF frame is not successfully captured:

RETURN None

RETURN frame

3. DEFINE preprocess(frame)

Resize frame to suitable resolution

Convert BGR image to grayscale

Apply Gaussian blur if required

RETURN processed frame

4. DEFINE detect_faces(gray_frame)

Apply face detection algorithm

Detect all faces

FOR each detected face:

Obtain:

x, y, width, height

Store bounding box

RETURN all bounding boxes

5. DEFINE recognize_face(gray_frame, face_box)

Extract face region:

face = gray_frame[y:y+h, x:x+w]

```
Resize face according to
recognition-model input size
Normalize the face image
Send face to recognition model
Compare with trained dataset
IF similarity/confidence > threshold:
    label = matched person's name
ELSE:
    label = "Unknown"
RETURN label
```

6. DEFINE draw_results(frame, face_box, label)

```
Draw bounding box using:
```

```
cv2.rectangle()
```

```
Draw name using:
```

```
cv2.putText()
```

```
RETURN frame
```

7. MAIN PROCESSING LOOP

```
WHILE camera is running:
```

```
    frame = capture_frame()
```

```
    IF frame == None:
```

```
        BREAK
```

```
    processed_frame = preprocess(frame)
```

```
    face_boxes = detect_faces(
        processed_frame
    )
```

```

FOR each face_box:
    label = recognize_face(
        processed_frame,
        face_box
    )
    frame = draw_results(
        frame,
        face_box,
        label
    )

```

Calculate FPS

Display:

Number of faces

FPS

Show frame using:

cv2.imshow()

IF user presses 'q' or ESC:

BREAK

8. RELEASE SYSTEM

cap.release()

cv2.destroyAllWindows()

END

4. OpenCV Implementation Concepts

A. Image Acquisition

cap = cv2.VideoCapture(0)

ret, frame = cap.read()

VideoCapture() continuously obtains frames from the webcam.

B. Preprocessing

```
gray = cv2.cvtColor(  
    frame,  
    cv2.COLOR_BGR2GRAY  
)  
gray = cv2.GaussianBlur(  
    gray,  
    (5, 5),  
    0  
)
```

Grayscale reduces computation, while Gaussian filtering reduces noise.

C. Face Detection

For a basic OpenCV implementation, Haar Cascade can be used:

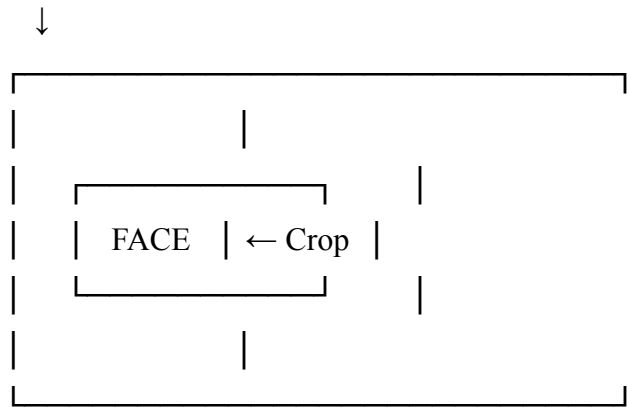
```
face_cascade = cv2.CascadeClassifier(  
    cv2.data.haarcascades +  
    "haarcascade_frontalface_default.xml"  
)  
  
faces = face_cascade.detectMultiScale(  
    gray,  
    scaleFactor=1.1,  
    minNeighbors=5,  
    minSize=(40, 40)  
)
```

The detector returns the coordinates of all detected faces.

5. Face Recognition

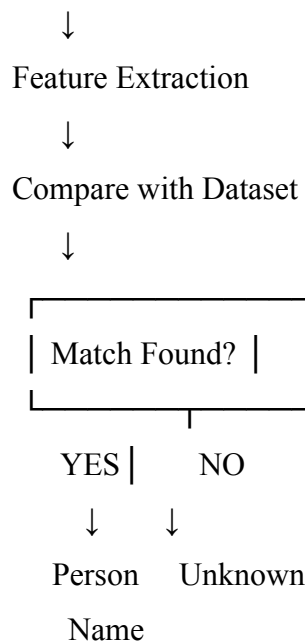
The detected face is cropped:

Full Frame



The cropped face is compared with the trained dataset.

Detected Face



A practical system can use LBPH, Eigenfaces/Fisherfaces, or a deep face-embedding model for recognition.

6. Multiple Face Handling

If three faces are detected:

Face 1 → Compare → Harini

Face 2 → Compare → Arun

Face 3 → Compare → Unknown

Each face is processed independently during the same frame.

7. Visualization

```
cv2.rectangle(  
    frame,  
    (x, y),  
    (x+w, y+h),  
    (0, 255, 0),  
    2  
)  
cv2.putText(  
    frame,  
    label,  
    (x, y-10),  
    cv2.FONT_HERSHEY_SIMPLEX,  
    0.7,  
    (0, 255, 0),  
    2  
)
```

Expected Output

```
|-----|  
| REAL-TIME FACE RECOGNITION |  
|                                     |  
|-----|-----|  
| | | | | | |  
| | FACE | | FACE | |  
| | | | | | |
```

Harini	Unknown		
Faces: 2	FPS: 30		

8. Efficient Loop Structure

For real-time performance:

Capture Frame



Resize



Detect Faces



FOR each face



Recognize only face ROI



Draw Results



Display



Next Frame

The entire frame should not be sent to the recognition model. Only detected face regions are processed.

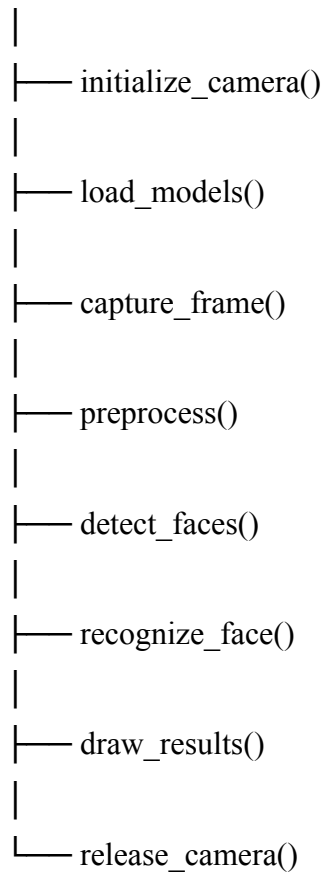
Performance Improvements

1. Resize frames before detection.
2. Use grayscale for face detection.
3. Process only detected face ROIs.

4. Use lightweight recognition models.
5. Avoid unnecessary image conversions.
6. Use GPU acceleration when available.
7. Use face tracking between detections to reduce repeated detection.
8. Monitor FPS to measure real-time performance.

9. Modular Design

main()



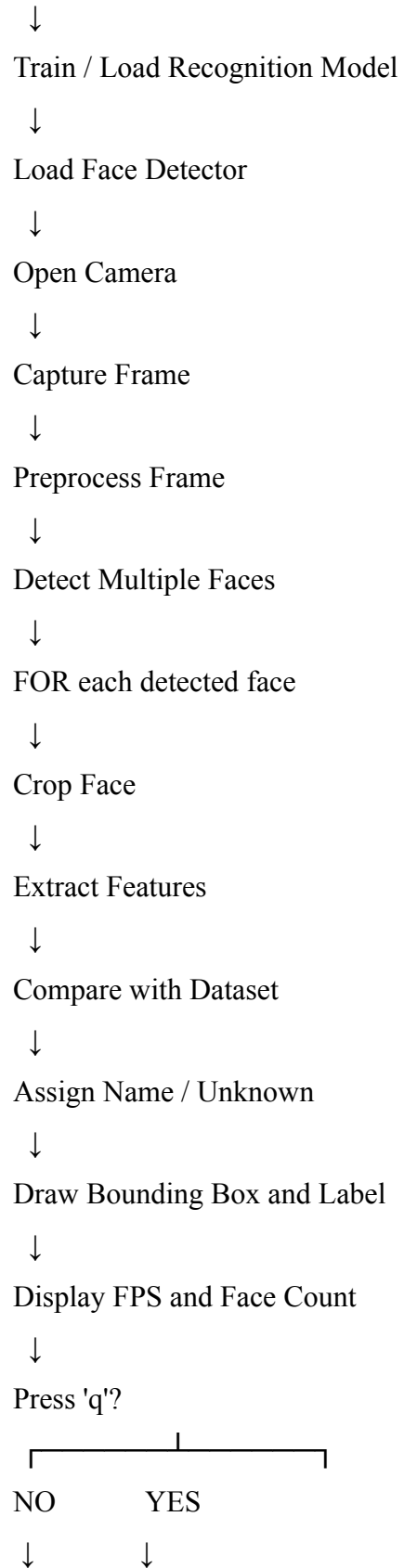
This modular structure makes the system easy to test, maintain, and upgrade.

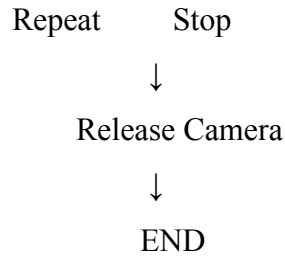
10. Complete Algorithm

START



Load Dataset





11. Conclusion

The proposed Real-Time Face Detection and Recognition Pipeline combines camera acquisition, preprocessing, face detection, face recognition, and visualization in a modular loop. Multiple faces can be detected and recognized independently in every frame. Processing only the detected face regions, resizing frames, using lightweight models, and optionally tracking faces between detections reduces processing delay and supports continuous real-time operation.

2. OCR Pipeline for Document Processing

A document processing system must:

- A. Read input images containing printed text
- B. Enhance image quality (noise removal, contrast improvement)
- C. Segment text regions
- D. Extract and display recognized text

Constraints:

- Input images may be noisy and low contrast
- Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

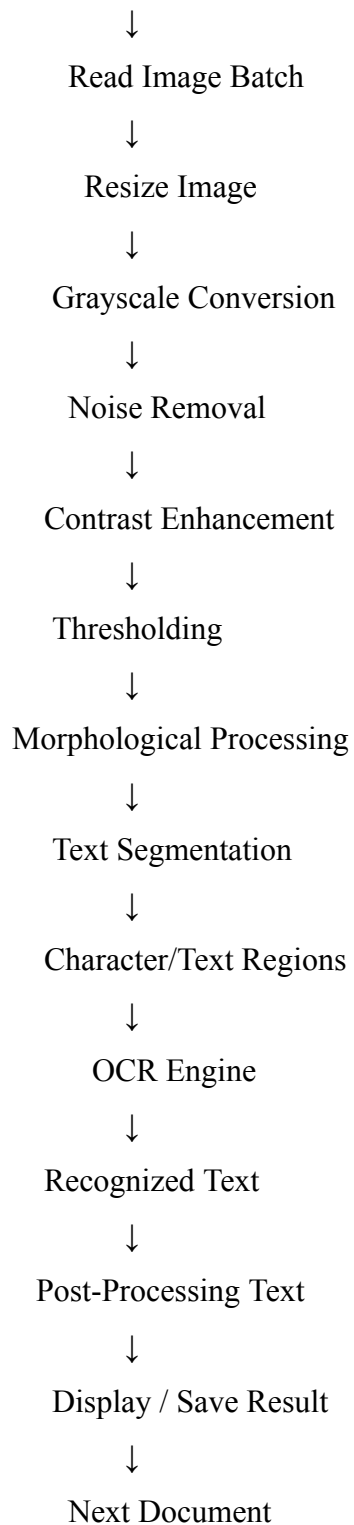
ANSWER:

1. Aim

To design an OpenCV-based OCR pipeline that reads multiple document images, removes noise, improves contrast, segments text regions, extracts text using OCR, and displays/saves the recognized text.

2. Block Diagram

Input Document Folder



3. Detailed Pseudocode

BEGIN

1. INITIALIZATION

Import OpenCV

Import OCR library

Define input folder

Define output folder

Get all image filenames
from input folder

2. DEFINE load_image(filename)

image ← cv2.imread(filename)

IF image is not available:

 RETURN failure

RETURN image

3. DEFINE preprocess(image)

Resize image to suitable size

Convert image from BGR to grayscale:

gray ← cv2.cvtColor(image, BGR2GRAY)

Remove noise:

gray ← cv2.GaussianBlur(gray, (5,5), 0)

Improve contrast:

enhanced $\leftarrow$ CLAHE(gray)

RETURN enhanced

4. DEFINE segment_text(enhanced)

IF illumination is uniform:

binary $\leftarrow$ cv2.threshold(
 enhanced,
 threshold_value,
 255,
 THRESH_BINARY
)

ELSE:

binary $\leftarrow$ cv2.adaptiveThreshold(
 enhanced,
 255,
 ADAPTIVE_THRESH_GAUSSIAN_C,
 THRESH_BINARY,
 block_size,
 constant
)

Create morphological kernel

Apply morphological opening
 to remove small noise

Apply morphological closing

to fill small gaps

Find contours:

contours $\leftarrow$ cv2.findContours(binary)

FOR each contour:

Calculate bounding rectangle

Filter regions using:

area

width

height

aspect ratio

Sort text regions:

Top-to-bottom

Left-to-right

RETURN text regions

5. DEFINE prepare_for_OCR(region)

Crop text region

Resize if required

Normalize intensity

Apply suitable threshold

RETURN OCR-ready region

6. DEFINE extract_text(text_regions)

recognized_text $\leftarrow$ empty string

FOR each region:

roi $\leftarrow$ prepare_for_OCR(region)

text $\leftarrow$ OCR(roi)

Append text to recognized_text

RETURN recognized_text

7. DEFINE postprocess(text)

Remove unnecessary spaces

Remove unwanted symbols
Correct line spacing
Preserve paragraph structure
RETURN cleaned_text

8. BATCH PROCESSING

FOR each image in input folder:
 image ← load_image(image)
 IF image loading fails:
 Continue to next image
 enhanced ← preprocess(image)
 regions ← segment_text(enhanced)
 text ← extract_text(regions)
 final_text ← postprocess(text)
 Display original image
 Display processed image
 Display recognized text
 Save final_text
 into output folder

9. TERMINATION

Display:
 "Batch processing completed"
Close all OpenCV windows

END

4. Preprocessing Stage

A. Resize

image = cv2.resize(image, (1000, 800))

Purpose: Standardizes images and reduces processing time.

B. Grayscale Conversion

```
gray = cv2.cvtColor(  
    image,  
    cv2.COLOR_BGR2GRAY  
)
```

Converts the three-channel color image into a single intensity channel.

C. Noise Removal

```
blur = cv2.GaussianBlur(  
    gray,  
    (5, 5),  
    0  
)
```

Gaussian filtering removes small noise and improves segmentation.

5. Contrast Enhancement

For low-contrast documents, CLAHE is useful.

```
clahe = cv2.createCLAHE(  
    clipLimit=2.0,  
    tileGridSize=(8, 8)  
)  
enhanced = clahe.apply(blur)
```

Why CLAHE?

It improves local contrast, making faint characters more visible even when illumination is not uniform.

Low Contrast Image

↓

CLAHE

↓

Improved Character Visibility

6. Text Segmentation

Thresholding separates text from the background.

Adaptive Thresholding

```
binary = cv2.adaptiveThreshold(  
    enhanced,  
    255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY,  
    11,  
    2  
)
```

Adaptive thresholding is preferred for documents with:

- Shadows
- Uneven lighting
- Dark and bright regions
- Background variations

7. Morphological Processing

```
kernel = cv2.getStructuringElement(  
    cv2.MORPH_RECT,  
    (3, 3)  
)
```

```
clean = cv2.morphologyEx(  
    binary,  
    cv2.MORPH_OPEN,  
    kernel
```

)

```
clean = cv2.morphologyEx(  
    clean,  
    cv2.MORPH_CLOSE,  
    kernel  
)
```

Opening

Removes small unwanted noise.

Closing

Fills small gaps in characters and text regions.

8. Text Region Detection

Contours are used to identify possible text regions.

```
contours, _ = cv2.findContours(  
    clean,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_SIMPLE  
)
```

For each contour:

IF area is too small:

Reject

IF width/height is unrealistic:

Reject

IF aspect ratio is unsuitable:

Reject

ELSE:

Keep as text region

The valid regions are sorted in reading order.

Region 1: Heading	
Region 2: First line	
Region 3: Second line	
Region 4: Third line	

9. OCR Stage

The segmented text regions are passed to an OCR engine such as Tesseract.

```
import pytesseract
```

```
text = pytesseract.image_to_string(  
    clean,  
    config="--psm 6"  
)
```

```
print(text)
```

The OCR engine converts the image characters into machine-readable text.

Image Characters

↓

OCR

↓
"SMART TRAFFIC
MONITORING SYSTEM"

10. Batch Processing

Since batch processing is required, all images in a folder are processed sequentially.

Input Folder

```
|
|— document1.jpg
|— document2.jpg
|— document3.jpg
|— document4.jpg
|— document5.jpg
```

↓

Process one by one

↓

Output Folder

```
|
|— document1.txt
|— document2.txt
|— document3.txt
|— document4.txt
|— document5.txt
```

11. OpenCV Functions Used

Stage	Function	Purpose
Image input	<code>cv2.imread()</code>	Read image

Resize	<code>cv2.resize()</code>	Standardize image
Grayscale	<code>cv2.cvtColor()</code>	Simplify processing
Noise removal	<code>cv2.GaussianBlur()</code>	Remove noise
Contrast	<code>cv2.createCLAHE()</code>	Improve readability
Threshold	<code>cv2.threshold()</code>	Binary segmentation
Adaptive threshold	<code>cv2.adaptiveThreshold()</code>	Handle uneven lighting
Morphology	<code>cv2.morphologyEx()</code>	Clean image
Contours	<code>cv2.findContours()</code>	Find text regions
Rectangle	<code>cv2.boundingRect()</code>	Extract regions
Display	<code>cv2.imshow()</code>	Show result

12. Complete Processing Flow

Original Document



Resize



Grayscale



Gaussian Blur



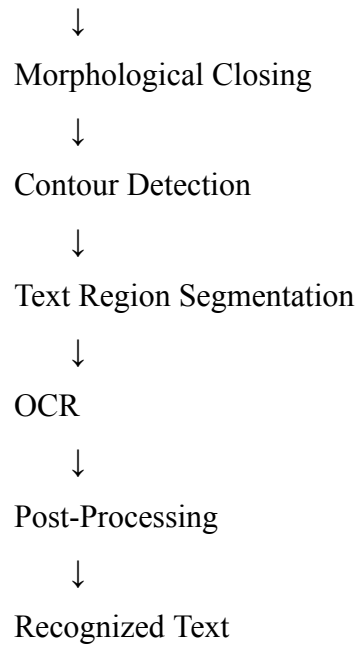
CLAHE Enhancement



Adaptive Threshold



Morphological Opening



13. Example Output

OCR RESULT	
Document: document1.jpg	
SMART TRAFFIC MONITORING SYSTEM	
Vehicle detection and tracking	
are performed using OpenCV.	
OCR Status: SUCCESS	

14. Accuracy and Performance

Accuracy

The system improves OCR accuracy by:

- Removing image noise.
- Enhancing contrast.
- Using adaptive thresholding.
- Removing small unwanted regions.
- Segmenting text before OCR.
- Resizing small text regions.

Performance

Real-time performance is not the primary constraint because this is a batch processing system.

However, performance can still be improved by:

- Resizing large images.
- Processing only required regions.
- Avoiding repeated image conversions.
- Processing images sequentially.

15. Conclusion

The proposed OpenCV OCR pipeline performs complete document processing using image acquisition, grayscale conversion, noise removal, CLAHE contrast enhancement, adaptive thresholding, morphological filtering, text segmentation, and OCR. The preprocessing stages improve readability of noisy and low-contrast documents, while batch processing allows multiple images to be automatically processed and their recognized text to be displayed or saved.

5. Facial Expression Recognition System

A mobile-based system must:

- A. Capture images from a camera
- B. Detect faces
- C. Extract facial features
- D. Classify expressions (happy, sad, neutral)
- E. Display results

Constraints:

- Limited computational resources
- Real-time response required

Write pseudocode for the pipeline, ensuring efficient processing and modular design for detection, feature extraction, and classification.

ANSWER:

Aim

To design a lightweight real-time system that detects faces and classifies expressions as Happy, Sad, or Neutral.

Block Diagram

Camera



Capture Frame



Resize + Grayscale



Face Detection



Face ROI



Feature Extraction



Expression Classifier



Happy / Sad / Neutral



Display Result



Next Frame

Pseudocode

BEGIN

Load lightweight face detection model

Load trained expression classification model

Open mobile camera

WHILE camera is running:

 Capture frame

 Resize frame for faster processing

 Convert frame to grayscale

 Detect all faces

 FOR each detected face:

 Crop face ROI

Resize and normalize face

Extract facial features

expression $\leftarrow$ Classifier(features)

IF expression = 0:

 label $\leftarrow$ "Happy"

ELSE IF expression = 1:

 label $\leftarrow$ "Sad"

ELSE:

 label $\leftarrow$ "Neutral"

Draw bounding box around face

Display expression label

Display processed frame

IF user exits:

 BREAK

Release camera

Close display

END

Efficiency Measures

- Use small/resized frames to reduce computation.
- Process only the face ROI, not the entire image.
- Use a lightweight CNN/model suitable for mobile devices.

- Avoid unnecessary image conversions.
- Use frame skipping or tracking when required for faster response.

Output: Live camera feed with a bounding box and expression label such as “Happy”, “Sad”, or “Neutral.”

12. Face Detection with ROI Extraction

A system must:

- A. Detect faces in an image
- B. Extract face regions
- C. Save cropped faces

Constraints:

- Handle multiple faces
- Avoid redundant processing

Write pseudocode including detection, ROI extraction, and saving logic.

ANSWER:

Aim

To detect multiple faces in an image, extract each face as a Region of Interest (ROI), and save the cropped faces efficiently.

Block Diagram

Input Image



Resize / Grayscale



Face Detection



Multiple Face Regions



ROI Extraction



Duplicate Check



Save Cropped Faces

Pseudocode

BEGIN

Load input image

Load pre-trained face detection model

Resize image if required

Convert image to grayscale

Detect all faces

Initialize face_count = 0

FOR each detected face (x, y, w, h):

 Check whether face ROI is valid

IF ROI is valid AND not already processed:

Extract ROI:

```
face = image[y:y+h, x:x+w]
```

Resize face if required

```
filename = "face_" + face_count + ".jpg"
```

Save face using cv2.imwrite()

```
face_count = face_count + 1
```

Display total number of faces

END

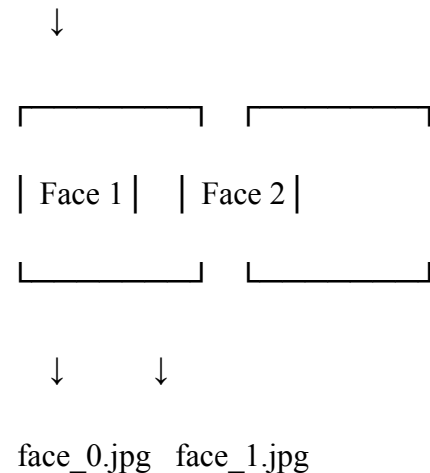
Main OpenCV Functions

Function	Purpose
<code>cv2.imread()</code>	Read image
<code>cv2.cvtColor()</code>	Convert to grayscale
<code>detectMultiScale()</code>	Detect multiple faces
<code>image[y:y+h,x:x+w]</code>]	Extract face ROI

<code>cv2.imwrite()</code>	Save cropped face
----------------------------	-------------------

Output

Input Image



Conclusion: The system efficiently detects multiple faces, extracts only the required ROIs, avoids redundant processing, and saves each detected face as a separate image.

13. Video Stabilization Pipeline

A system must:

- Read video frames
- Detect motion between frames
- Stabilize output

Constraints:

- Maintain real-time processing
- Reduce jitter

Write pseudocode including motion estimation, transformation, and display.

ANSWER:

Aim

To stabilize a video by detecting motion between consecutive frames, estimating camera movement, and applying a corrective transformation to reduce jitter.

Block Diagram

Video Input



Read Frames



Grayscale Conversion



Detect Feature Points



Track Features



Estimate Motion



Smooth Motion



Apply Transformation



Crop / Border Correction



Display Stabilized Video

Pseudocode

BEGIN

Open video using cv2.VideoCapture()

Read first frame

Convert first frame to grayscale

Detect feature points

WHILE video contains frames:

 Read current frame

 Convert current frame to grayscale

 Track feature points between

 previous and current frames

 Estimate transformation

 (translation + rotation)

 Calculate camera motion

 Smooth motion to reduce jitter

Apply inverse transformation

using `cv2.warpAffine()`

Crop unwanted borders

Display stabilized frame

Update previous frame

Update feature points

IF user presses 'q':

BREAK

Release video

Close all windows

END

Important OpenCV Functions

Function	Purpose
<code>cv2.VideoCapture()</code>	Read video
<code>cv2.cvtColor()</code>	Convert to grayscale
<code>cv2.goodFeaturesToTrack()</code>	Detect feature points

<code>cv2.calcOpticalFlowPyrLK()</code>	Track motion
<code>cv2.estimateAffinePartial2D()</code>	Estimate transformation
<code>cv2.warpAffine()</code>	Apply stabilization
<code>cv2.imshow()</code>	Display output

Real-Time Optimization

- Resize frames before processing.
- Track a limited number of feature points.
- Detect new features only when necessary.
- Use lightweight affine transformation.
- Smooth motion over consecutive frames.

Output

Unstable Video



Motion Estimation



Motion Smoothing



Inverse Transformation



Stable Video

Conclusion: The pipeline reduces camera shake and jitter by estimating frame-to-frame motion, smoothing the camera trajectory, and applying the inverse transformation while maintaining real-time performance.

Code of Conduct:

*I _HARITHI S (192421151)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

HARITHI S

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach

Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient